

horizontal segment of a net from the bottom in the same column. In Fig. 6.5(a), this vertical constraint assigns the horizontal segment of net A to a track above the horizontal segment of net B . To satisfy these constraints, at least three columns are required to “uncross” the two nets.

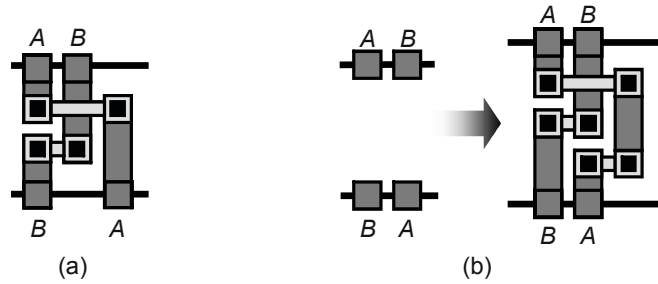


Fig. 6.5 Examples of vertically constrained nets. (a) Nets A and B do not have a vertical conflict. (b) Nets A and B have a vertical conflict. The two nets can be routed by splitting the vertical segment of one net and using an additional third track.

Although a vertical constraint implies a horizontal constraint, the converse is not necessarily true. However, both types of constraints must be satisfied when assigning segments in a channel.

6.2 Horizontal and Vertical Constraint Graphs

The relative positions of nets in a channel routing instance, encoded with horizontal and vertical constraints, can be modeled by *horizontal* and *vertical constraint graphs*, respectively. These graphs are used to (1) initially predict the minimum number of tracks that are required and (2) detect potential routing conflicts.

► 6.2.1 Horizontal Constraint Graphs

Zone representation. In a channel, all horizontal wire segments must span at least the leftmost and rightmost pins of their respective nets. Let $S(col)$ denote the set of nets that pass through column col . In other words, $S(col)$ contains all nets that either (1) are connected to a pin in column col or (2) have pin connections to both the left and right of col . Since horizontal segments cannot overlap, each net in $S(col)$ must be assigned to a different track in column col . Only a subset of all columns is needed to describe the entire channel. If there exist columns i and j such that $S(i)$ is a subset of $S(j)$, then $S(i)$ can be ignored since it imposes fewer constraints on the routing solution than $S(j)$. In Fig. 6.6, every $S(col)$ is a subset of at least one of $S(c)$, $S(f)$, $S(g)$ or $S(i)$. Furthermore, the *maximal* columns c , f , g and i comprise a minimal set of columns with this property. Note that these columns together contain all the nets.

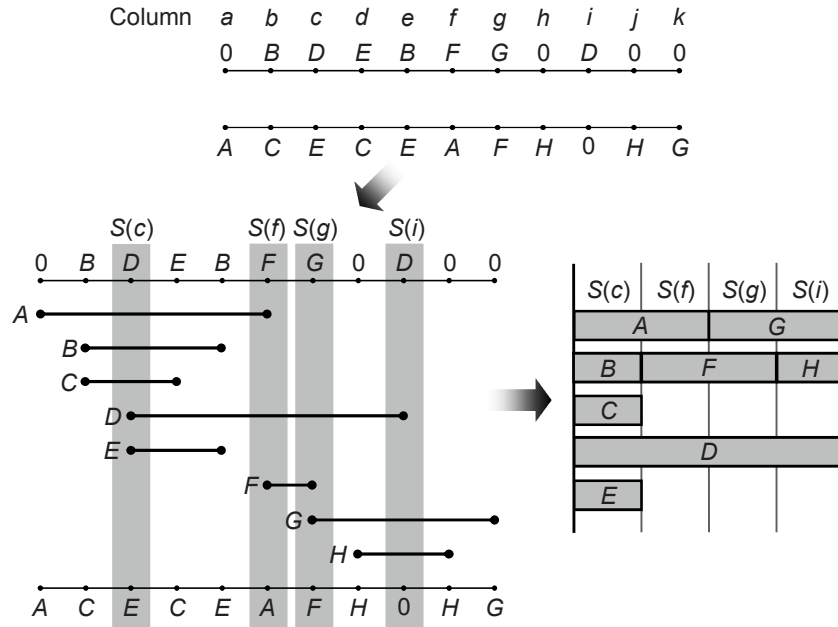


Fig. 6.6 A channel routing problem (top) and its corresponding zone representation (right). Only the maximal columns are shown.

Graphical representation. The nets within the channel can also be represented by a *horizontal constraint graph* $HCG(V, E)$, where nodes $v \in V$ correspond to the nets of the netlist and an undirected edge $e(i, j) \in E$ exists between nodes i and j if the corresponding nets are both elements of some set $S(col)$. In other words, $e \in E$ if the corresponding nets are horizontally constrained. Fig. 6.7 illustrates the HCG for the channel routing instance of Fig. 6.6. A lower bound on the number of tracks required by the channel routing can be found from either the HCG or the zone representation. This lower bound is given by the maximum cardinality of any $S(col)$.

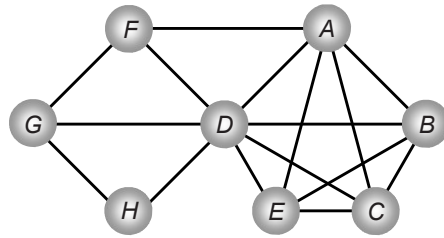


Fig. 6.7 The HCG for the channel routing instance of Fig. 6.6. With either the HCG or the zone representation, a lower bound on the number of tracks is five.

► 6.2.2 Vertical Constraint Graphs

Vertical constraints are represented by a *vertical constraint graph* $VCG(V, E)$. A node $v \in V$ represents a net. A directed edge $e(i, j) \in E$ connects nodes i and j if net i

must be located above net j . However, an edge that can be derived by transitivity is not included. For instance, in Fig. 6.8, edge (B,C) is not included because it can be derived from edges (B,E) and (E,C) .

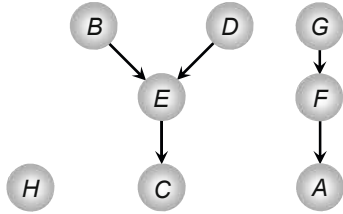


Fig. 6.8 The VCG for the channel routing instance of Fig. 6.6.

A cycle in the VCG indicates a conflict where vertical segments of two nets overlap at a specific column. That is, the horizontal segments of the two nets would have to be simultaneously above and below each other. This contradiction can be resolved by splitting the net and using an additional track (Fig. 6.9).

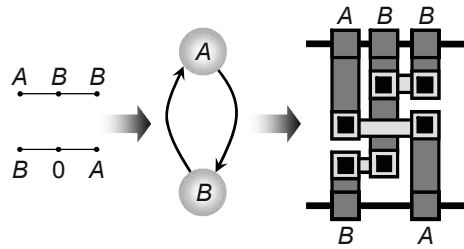


Fig. 6.9 A channel routing problem (left), its VCG with a cycle or conflict (center), and a possible solution using net splitting and an additional third track (right).

If a cycle occurs in a maximum-cardinality set $S(col)$ of the HCG, the lower bound on the number of required tracks, based on the number of nets in $S(col)$, is no longer tight. Nets must now be split and the minimum number of required tracks must be adjusted to account for both HCG and VCG conflicts.

Example: Vertical and Horizontal Constraint Graphs

Given: channel routing instance (right).

Task: find the horizontal constraint graph (HCG) and the vertical constraint graph (VCG) of nets A - F .

Column

	a	b	c	d	e	f	g
0	B	D	D	B	A	C	E
	D	C	E	F	0	A	F

Solution:

TOP and BOT vectors: $TOP = [0\ B\ D\ B\ A\ C\ E]$, $BOT = [D\ C\ E\ F\ 0\ A\ F]$

Determine $S(col)$ for $col = a \dots g$.

$$\begin{aligned} S(a) &= \{D\} & S(b) &= \{B, C, D\} & S(c) &= \{B, C, D, E\} & S(d) &= \{B, C, E, F\} \\ S(e) &= \{A, C, E, F\} & S(f) &= \{A, C, E, F\} & S(g) &= \{E, F\} \end{aligned}$$

Find the maximal $S(col)$.

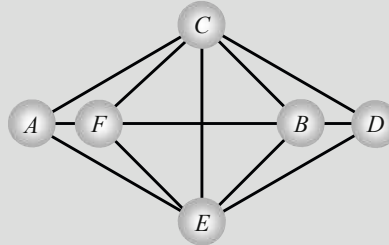
$S(a) = \{D\}$ and $S(b) = \{B, C, D\}$ are both subsets of $S(c) = \{B, C, D, E\}$.

$S(f) = \{A, C, E, F\}$ and $S(g) = \{E, F\}$ are both subsets of $S(e) = \{A, C, E, F\}$.

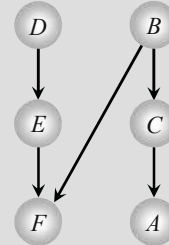
Maximal $S(col)$: $S(c) = \{B, C, D, E\}$ $S(d) = \{B, C, E, F\}$ $S(e) = \{A, C, E, F\}$

Find the HCG and VCG.

$S(c)$	$S(d)$	$S(e)$
B		A
	C	
D		F
	E	



HCG



VCG

Since there are no cycles in the VCG, no *net splitting* is required (Sec. 6.2.2). Thus, each net only needs one horizontal segment for routing.

The track assignment is based on both the VCG and the HCG. For instance, based on the VCG, net D is assigned to the topmost track. The other net that is on the top level in the VCG (net B) is assigned to another track due to the HCG.

The HCG determines a lower bound on the number of required tracks. Since there are no cycles (conflicts) in the VCG, the minimum number of tracks is equal to the cardinality of the largest $S(col)$, here being $|S(c)| = |S(d)| = |S(e)| = 4$.

6.3 Channel Routing Algorithms

6.3

Channel routing seeks to minimize the number of tracks required to complete routing. In gate-array designs, channel height is typically fixed, and algorithms are designed to pursue 100% routing completion.

► 6.3.1 Left-Edge Algorithm

An early channel routing algorithm was developed by *Hashimoto* and *Stevens* [6.8]. Their simple and widely used *left-edge* heuristic, based on the VCG and the zone representation, *greedily maximizes the usage of each track*. The former identifies the assignment order of nets to tracks, and the latter determines which nets may share the same track. *Each net uses only one horizontal segment (trunk)*.

The left-edge algorithm works as follows. Start with the topmost track (line 1). For all unassigned nets *nets_unassigned* (line 3), generate the VCG and the zone representation (lines 4-5). Then, in left-to-right order (line 6), for each unassigned net n , assign it to the current track if (1) n has no predecessors in the VCG and (2) it does not cause a conflict with any nets that have been previously assigned (lines 7-11). Once n has been assigned, remove it from *nets_unassigned* (line 12). After all unassigned nets have been considered, increment the track index (line 13). Continue this process until all nets have been assigned to routing tracks (lines 3-13).

Left-Edge Algorithm**Input:** channel routing instance CR **Output:** track assignments for each net

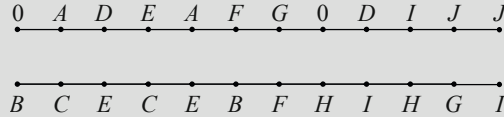
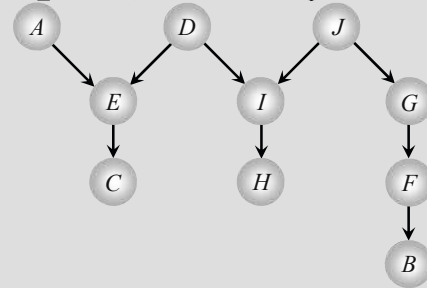
```

1.   $curr\_track = 1$  // start with topmost track
2.   $nets\_unassigned = Netlist$ 
3.  while ( $nets\_unassigned \neq \emptyset$ ) // while nets still unassigned
4.     $VCG = VCG(CR)$  // generate VCG and zone
5.     $ZR = ZONE\_REP(CR)$  // representation
6.     $SORT(nets\_unassigned, start\ column)$  // find left-to-right ordering
    // of all unassigned nets

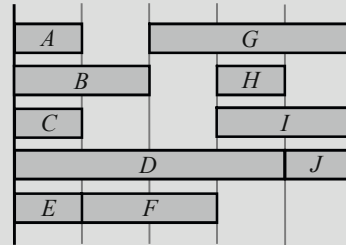
7.    for ( $i = 1$  to  $|nets\_unassigned|$ )
8.       $curr\_net = nets\_unassigned[i]$ 
9.      if ( $PARENTS(curr\_net) == \emptyset \ \&\&$  // if  $curr\_net$  has no parent
10.         ( $TRY\_ASSIGN(curr\_net, curr\_track)$ ) // and does not cause
    // conflicts on  $curr\_track$ ,
11.          $ASSIGN(curr\_net, curr\_track)$  // assign  $curr\_net$ 
12.          $REMOVE(nets\_unassigned, curr\_net)$ 
13.      $curr\_track = curr\_track + 1$  // consider next track

```

The left-edge algorithm finds a solution with the minimum number of tracks, or the maximum cardinality of $S(col)$, if there are no cycles in the VCG. *Yoshimura* [6.18] enhanced track selection by incorporating net length in the VCG. *Yoshimura* and *Kuh* [6.19] improved track utilization by net splitting before constructing the VCG.

Example: Left-Edge Algorithm**Given:** channel routing instance (right).**Task:** use the left-edge algorithm to route nets A - J in the channel.**Solution:** $curr_track = 1$, VCG and zone representation:

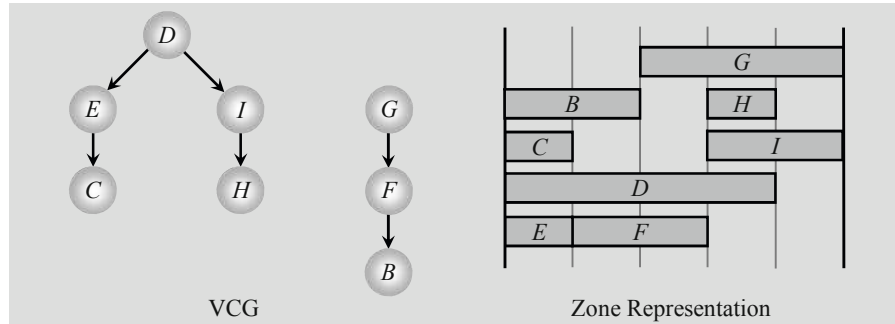
VCG



Zone Representation

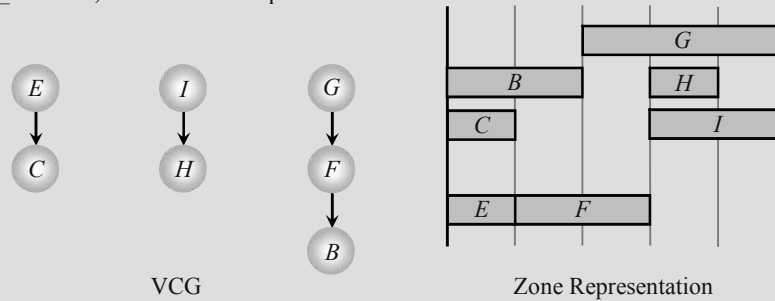
Nets A , D and J have no predecessors in the VCG and can be assigned to $curr_track = 1$. Net A is assigned first because it is the leftmost. Net D can no longer be assigned to $curr_track$ because of a conflict with net A . However, net J can be assigned to $curr_track$, since there is no conflict. Remove nets A and J from the VCG and the zone representation.

 $curr_track = 2$, VCG and zone representation:



Nets *D* and *G* have no predecessors in the VCG and can be assigned to $curr_track = 2$. Net *D* is assigned first because it is the leftmost. Net *G* can no longer be assigned to $curr_track$ because of a conflict with net *D*. Remove net *D* from the VCG and the zone representation.

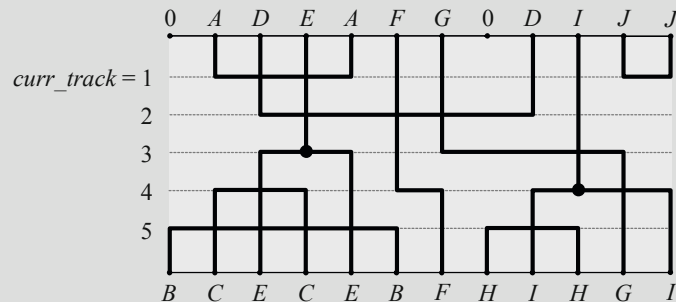
$curr_track = 3$, VCG and zone representation:



Nets *E*, *G* and *I* have no predecessors in the VCG and can be assigned to $curr_track = 3$. Net *E* is assigned first because it is the leftmost. Net *G* is assigned to $curr_track$ because it does not conflict with any nets. Net *I* cannot be assigned because of a conflict with net *G*. Remove nets *E* and *G* from the VCG and the zone representation.

$curr_track = 4$ and $curr_track = 5$ are similar to previous iterations. Nets *C*, *F* and *I* are assigned to track 4. Nets *B* and *H* are assigned to track 5.

Routed channel:



► 6.3.2 Dogleg Routing

To deal with cycles in the VCG, a *dogleg* (L-shape) can be introduced. Doglegs not only alleviate conflicts in VCGs (Fig. 6.10), but also help reduce the total number of tracks (Fig. 6.11).

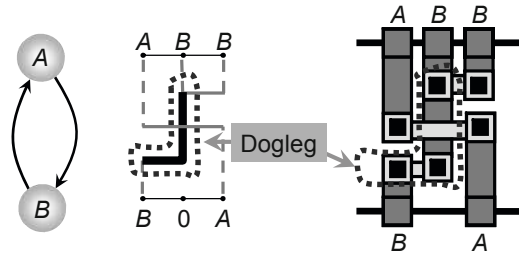


Fig. 6.10 A dogleg is introduced to route around the conflict for net B.

The dogleg algorithm, developed by *Deutsch* [6.6] in the 1970s, eliminates cycles in VCGs and reduces the number of routing tracks (Fig. 6.11). The algorithm extends the left-edge algorithm by splitting p -pin nets ($p > 2$) into $p - 1$ horizontal segments. Net splitting to introduce doglegs occurs only in columns that contain a pin of the given net, under the assumption that additional vertical tracks are not available. After net splitting, the algorithm follows the left-edge algorithm (Sec. 6.3.1). The subnets are represented by the VCG and zone representation.

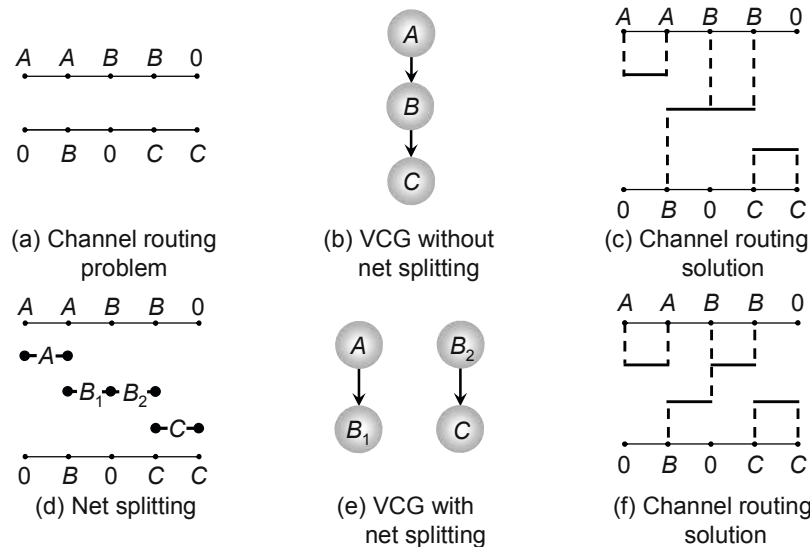


Fig. 6.11 Example showing how net splitting can reduce the number of required tracks. (a) The channel routing instance. (b) The conventional VCG without net splitting. (c) The channel routing solution without net splitting uses three tracks. (d) Net splitting applied to the channel routing solution from (c). (e) The new VCG after net splitting. (f) The new channel routing solution with net splitting uses only two tracks.

Example: Dogleg Left-Edge Algorithm**Given:** channel routing instance (right).**Task:** use the dogleg left-edge algorithm to route the nets A - D in the channel.

Column	a	b	c	d	e	f
	C	D	0	D	A	A

	B	B	C	0	C	D
--	-----	-----	-----	-----	-----	-----

Solution:

Split the nets, find $S(col)$ and determine the zone representation. Note: subnets of a net can be placed on the same track regardless of overlap in the zone representation.

a	b	c	d	e	f
C	D	0	D	A	A



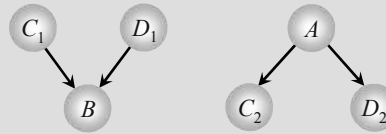
B	B	C	0	C	D
-----	-----	-----	-----	-----	-----

Net Splitting

$$\begin{aligned}
 S(a) &= \{B, C_1\} \\
 S(b) &= \{B, C_1, D_1\} \\
 S(c) &= \{C_1, C_2, D_1\} \\
 S(d) &= \{C_2, D_1, D_2\} \\
 S(e) &= \{A, C_2, D_2\} \\
 S(f) &= \{A, D_2\}
 \end{aligned}$$

 $S(col)$

Find the VCG.



Track assignment:

 $curr_track = 1$

Consider nets C_1 , D_1 and A . Assign net C_1 first, since it is the leftmost net in the zone representation. Of the remaining nets, only net A does not cause a conflict. Therefore, assign net A to $curr_track$. Remove nets C_1 and A from the VCG.

 $curr_track = 2$

Consider nets D_1 , C_2 and D_2 . Assign net D_1 first, since it is the leftmost net in the zone representation. Of the remaining nets, only net D_2 does not cause a conflict. Therefore, assign net D_2 to $curr_track$. Remove nets D_1 and D_2 from the VCG.

 $curr_track = 3$

Consider nets B and C_2 . Assign net B first, since it is the leftmost net in the zone representation. Net C_2 does not cause a conflict. Therefore, assign net C_2 to $curr_track$.

Routed channel:

